

eMQTT-AC: Reducing MQTT Tail Latency with eBPF-based Adaptive Control

ABSTRACT

In large-scale IoT deployments, MQTT is the de facto standard; however, under high-density traffic—where many devices publish at high rates simultaneously—network congestion can inflate tail latency to multi-second ranges. Existing approaches struggle to control such extreme conditions and often incur high integration cost. We propose eMQTT-AC, an application-transparent, kernel-signal-driven adaptive control system deployable at edge gateways. Using eBPF to observe TCP kernel signals (RTT, retransmissions, and buffer occupancy) in real time, eMQTT-AC dynamically adjusts publishers' send rate and batch size to preemptively suppress queue buildup at congestion onset. In evaluation with 20 publishers generating 1,000 messages per second, eMQTT-AC reduced the 95th percentile end-to-end latency from 1,717.90 ms to 464.61 ms (−73.0%) and the 99th percentile from 2,088.54 ms to 473.50 ms (−77.3%). The mean end-to-end latency decreased by 74.9% (1,226.69 ms → 307.37 ms), and the standard deviation dropped by 55.4% (402.61 ms → 179.63 ms), indicating markedly improved stability. Importantly, throughput slightly increased from 32.28 to 32.36 msgs/s, and CPU overhead remained close to 2%, confirming that eMQTT-AC is lightweight and practical for deployment on commodity edge gateways.

Key Words: MQTT, eBPF, tail latency, adaptive control, TCP kernel signals, edge gateway.

I . Introduction

The rapid proliferation of the Internet of Things (IoT) has amplified the importance of lightweight messaging protocols that collect and disseminate telemetry from vast numbers of devices. Among them, MQTT (Message Queuing Telemetry Transport) has become the de facto standard in resource-constrained environments thanks to its publish/subscribe model and low protocol overhead¹.

However, modern deployments—such as smart factories and connected-vehicle infrastructures—increasingly operate in *edge computing* settings where thousands of devices can simultaneously publish at hundreds to thousands of messages per second². In these high-density, high-load regimes, MQTT systems face a persistent challenge: *tail latency*. Even when average latency remains moderate, transient network degradations or broker bottlenecks can inflate the latency of a subset of messages into the multi-second range, undermining real-time control loops and eroding system reliability.

The problem of tail latency is well-known across various distributed systems and applications, from web services to cloud databases³, and its impact is particularly severe in edge environments where timely decision-making is critical.

Edge environments exacerbate this problem. Gateways bridging local operational networks and the cloud often have limited compute budgets, bandwidth asymmetry on uplinks, and heterogeneous last-hop links (wired, Wi-Fi, cellular). Moreover, regulatory and privacy constraints frequently require data locality and *site-by-site* control, which makes heavyweight broker modifications or cloud-dependent mitigations impractical. Consequently, any tail-latency remedy must be (i) application-transparent, (ii) broker-agnostic, (iii) lightweight enough for commodity edge gateways, and (iv) deployable incrementally without disrupting existing traffic. These constraints often lead to significant challenges in resource management, task offloading, and security in edge computing scenarios^{Dong2024TaskOS}.

Prior approaches attempt to mitigate tails by adding client-side hedging or retry logic, or by replacing TCP with alternative transports such as QUIC⁴. While effective in specific contexts, the former increases client complexity and overhead across a large device fleet, and the latter incurs infrastructure change costs and compatibility risks across diverse broker implementations and network middleboxes. Both are brittle under

rapidly shifting load and heterogeneous edge links.

To address these limitations, we propose **eMQTT-AC**, an eBPF-based *adaptive control* system that can be deployed transparently on edge gateways without modifying MQTT applications or broker code. Leveraging Linux eBPF, eMQTT-AC observes congestion signals directly from the broker host's TCP stack—including smoothed RTT (*srtt_us*), interval retransmissions, and send/receive buffer occupancy—at fine time granularity. A user-space controller applies EWMA smoothing, multi-signal gating, and hysteresis with hold times and cool-down to avoid control flapping. Upon detecting incipient congestion, the controller proactively publishes JSON commands on a dedicated MQTT *control topic* to adjust publishers' send rate and batch size (and, under severe pressure, QoS level). This closes the loop *at the edge*, suppressing early queue buildup before tails materialize, while preserving broker portability and application transparency. This approach aligns with the growing trend of using eBPF for advanced network monitoring, analysis, and adaptive traffic management, offering unparalleled visibility and control within the kernel without requiring kernel module modifications⁵.

Contributions. This paper makes the following contributions:

- We introduce a broker-agnostic, application-transparent control plane for MQTT tail-latency mitigation that leverages kernel TCP signals available via eBPF, suitable for resource-constrained edge gateways.
- We design a robust policy that fuses RTT, retransmission, and buffer occupancy with EWMA smoothing and hysteresis/hold-time mechanisms to prevent oscillation under bursty loads.
- We implement eMQTT-AC on commodity Linux and demonstrate, in a 20-publisher (1,000 msg/s) scenario, substantial reductions in p95/p99 latency with negligible CPU overhead and minimal throughput impact.

II . Related Work

2.1 MQTT and Tail Latency

MQTT is a lightweight publish/subscribe protocol brokered through a central *broker*. Along the end-to-end path, messages traverse the publisher's kernel send buffer (*sk_wmem_queued*), the network, broker-side internal queues (*ingress/topic/session*), and the

Table 1. Tail-latency mitigation strategies.

Strategy	Change scope	Actuator (what is tuned)	Roll-out
Client hedging/backoff	Clients	Redundancy / retry / backoff	Hard (fleet)
Broker internals	Broker	Queues / scheduling / backpressure	Medium (vendor)
MQTT over QUIC ^{4,6}	Clients + Broker	Transport stack (QUIC mapping)	Hard (compat.)
DRL for CC (transport-layer) ⁷	Kernel / Transport	TCP params (cwnd, pacing, RTO)	Research
eMQTT-AC (ours)	None (apps/broker)	Publisher rate / batch (+QoS)	Easy (edge)

subscriber’s kernel receive buffer (`sk_rmem_alloc`). Under high load, saturation at any of these queues causes backlog accumulation, inflating not only the mean but also the higher percentiles (p95/p99) of latency—i.e., *tail latency*³.

MQTT’s reliability modes require acknowledgment exchanges—QoS 1 (PUBLISH → PUBACK) and QoS 2 (PUBLISH → PUBREC → PUBREL → PUBCOMP). Publishers maintain unacknowledged messages in an *inflight window* and retransmit on loss. When queues are already growing, these protocol-level handshakes and retransmissions amplify delay and widen the tail¹. While QoS levels 1 and 2 aim to ensure message delivery, they can introduce significant overhead and increased latency, especially in congested networks due to retransmissions and additional handshakes⁸. Scaling MQTT brokers to handle thousands of concurrent connections and high message rates also presents challenges, often resulting in performance variability across different broker implementations and under various workloads⁸. Because MQTT typically runs over TCP, packet losses induce transport-layer head-of-line (HoL) blocking: bytes behind a missing segment stall until recovery. While QUIC mitigates transport-level HoL via multiplexed streams⁴, mapping MQTT semantics (ordering and QoS exchanges) to QUIC and ensuring middlebox compatibility remain non-trivial⁶.

Recent benchmarking of open-source MQTT brokers highlights substantial performance variability across implementations and workloads, underscoring the sensitivity of tail behavior to internal queueing and scheduling policies⁹. In experimental studies and emulations, NetEm is widely used to inject controlled impairments (loss, delay, jitter) to reproduce tail-inducing conditions and assess mitigation techniques¹⁰.

2.2 eBPF and Kernel Instrumentation

eBPF is a kernel virtual machine that safely executes verified bytecode on event hooks without modifying kernel code. Programs are written in a C-like language, verified for safety, JIT-compiled, and attached to hooks such as tracepoints, kprobes, and XDP. BPF maps provide lock-free data structures for exporting per-event or

per-flow state to user space.

For transport-level observability relevant to MQTT over TCP, tracepoints and kprobes offer a precise vantage point with low overhead. In particular, the `tcp_retransmit_skb` tracepoint exposes retransmission events; a kprobe on `tcp_sendmsg` observes the enqueue pressure via `sk_wmem_queued`; and a kprobe on `tcp_rcv_established` reads smoothed RTT (`srtt_us`) and the kernel receive-buffer occupancy (`sk_rmem_alloc`). These signals capture the onset of congestion from both sender and receiver sides at fine time granularity, enabling control loops in user space. The capabilities of eBPF extend beyond mere observation to powerful network monitoring, analysis, and even adaptive traffic control mechanisms, offering unparalleled visibility and control within the kernel without requiring kernel module modifications^{5,11}. Contemporary studies further characterize what in-kernel observability can (and cannot) capture for latency-sensitive request metrics⁵; practical references consolidate modern eBPF design and deployment patterns at scale¹¹. Unlike XDP-based datapath transformations, this line of work *does not* alter packet forwarding; it focuses on safe, low-intrusive observability and kernel-signal export for application-level control.

2.3 Tail-latency Mitigation Across the Stack

Tail latency in large-scale systems has been studied extensively, with seminal work highlighting its profound impact on overall system performance and reliability³. Techniques span multiple layers:

Client-side redundancy and backoff. Hedging—sending duplicate requests and accepting the first response—as well as exponential backoff and retry policies can reduce tails observed by clients. However, these approaches increase client complexity and fleet-wide overhead, and can create correlated load spikes under common-mode slowdowns, potentially exacerbating congestion rather than alleviating it.

Broker-level queueing and scheduling. Broker internals (accept queues, topic/session queues, backpres-

sure propagation) critically shape tail behavior. While implementation improvements help, they are broker-specific and not always feasible in brownfield deployments. Research into advanced scheduling algorithms for latency-critical workloads in data centers provides a broader context for optimizing queueing mechanisms. A recent gateway design, SegaNet, shows that priority-aware batching and NIC-level queueing can reduce p99 message latency from multi-second ranges to sub-second under load¹².

Transport evolution (MQTT over QUIC). While QUIC's multiplexed streams remove transport-level HoL and offer per-stream flow control⁴, recent evaluations reveal mixed results in high-speed and diverse environments: performance varies widely across implementations and network regimes¹³, and QUIC can even underperform TCP in ultra-fast Internet settings without careful tuning¹⁴. Hence, MQTT-over-QUIC remains promising⁶ but requires careful mapping of MQTT semantics and operator validation across real deployments.

Kernel/network control. TCP congestion control (e.g., CUBIC, BBR) optimizes throughput/fairness but does not directly target application-level p95/p99. Active queue management and pacing can blunt queue growth, but visibility into application semantics (topics, QoS) is limited at the transport layer. Notably, advanced congestion control algorithms like BBR aim to reduce queuing delay by optimizing for bandwidth-delay product rather than packet loss¹⁵, offering improvements at the transport layer that can complement higher-level controls. Other variants like CUBIC and Datacenter TCP also focus on optimizing throughput and latency in specific network conditions^{16,17}. Recent surveys also provide a good overview of advancements in TCP congestion control^{15,16,17}.

Learning-based control. Deep reinforcement learning (DRL) has been explored for congestion control and traffic management⁷. DRL agents operate on transport metrics and learn policies that improve link utilization and fairness. However, training complexity, stability, and generalization across heterogeneous edge links pose challenges, and most DRL work modulates transport parameters rather than application-facing rate/batch controls. Beyond transport, DRL-driven configuration search at the edge/cloud boundary (e.g., EdgeTuner) has shown substantial p99 reductions by rapidly converging to low-tail scheduler settings¹⁸.

2.4 Positioning of Our Approach

Our work is orthogonal to the above lines: rather than modifying clients, brokers, or transports, we retain the standard MQTT-over-TCP stack and form a closed loop at the *edge gateway*. By leveraging eBPF to export kernel TCP signals (EWMA RTT, retransmissions, buffer occupancy) and by driving *application-level* publisher controls (rate, batch, and optionally QoS) via a control topic, our approach directly targets incipient queue buildup that leads to tails, while remaining application-transparent and broker-agnostic. This distinction is crucial, as many prior solutions involve modifying core network components or application logic, which increases deployment complexity, especially in resource-constrained edge environments where resource allocation and management are critical challenges¹⁹. Furthermore, the ability to operate without application changes simplifies deployment in existing IoT infrastructures, aligning with the needs of diverse and often sensitive IoT analytics at the edge²⁰.

In summary, prior work either increases client complexity, requires broker or transport changes, or optimizes for throughput/fairness rather than application-visible p95/p99. By contrast, our edge-centric, kernel-signal-driven control plane directly suppresses early queue buildup without altering existing MQTT applications or brokers, and can be deployed incrementally alongside standard impairment emulation with NetEm¹⁰.

III. Methodology

3.1 System architecture

The controller is co-located on the broker host (edge gateway) and runs in the host operating system. The data plane remains unchanged: publishers send MQTT over TCP to the broker and subscribers consume as usual. A separate control plane is used for feedback: the controller publishes JSON commands on a dedicated MQTT control topic (for example, *control/room1*). Publishers subscribe to this topic and, upon receiving a command, adjust their send rate in hertz and their batch size in messages; under severe pressure the controller may request a lower QoS level. A minimum cool-down interval prevents command bursts, and an observe-only mode can be enabled to collect telemetry without issuing control actions.

Table 2. Control parameters and defaults.

Parameter (env)	Meaning	Default
TH_RTT_US	Base RTT threshold (microseconds)	60000
HI_RTT_US, LO_RTT_US	Hysteresis high/low bounds (microseconds)	70000 / 50000
EWMA_ALPHA	Smoothing factor in Eq. 2	0.3
TH_SNDBUF, TH_RCVBUF	Normalization caps for buffers (bytes)	262144 / 262144
SND_RATIO_HI, SND_RATIO_LO	Buffer thresholds (θ_H, θ_L)	0.85 / 0.60
T_HOLD_ON, T_HOLD_OFF	Hold times for state transitions (seconds)	0.8 / 1.5
INTERVAL_S	Aggregation period T (seconds)	2.0
CONTROL_MIN_SEC	Minimum interval between commands (seconds)	2.0
CTRL_RATE, CTRL_BATCH	Default send rate (Hz) and batch size (messages)	5 / 5
EDA_OBSERVE	Observe-only mode (1 enables)	0
MQTT_HOST, MQTT_PORT	Control channel endpoint	127.0.0.1 / 23232
CONTROL_TOPIC	Control topic name	control/room1
USE_ADAPTIVE, ADAPT_WIN	Optional adaptive RTT bounds, window size	0, 200

3.2 Kernel-signal collection via eBPF

We attach lightweight probes on the broker host and track only flows for the MQTT port. The tracepoint `tcp_retransmit_skb` increments a per-flow cumulative retransmission counter. A kprobe on `tcp_sendmsg` reads the kernel send-buffer occupancy (`sk_wmem_queued`). A kprobe on `tcp_rcv_established` reads the smoothed RTT (`srtt_us`) and the kernel receive-buffer occupancy (`sk_rmem_alloc`). Per-flow statistics are kept in a BPF hash map keyed by IPv4 addresses and TCP ports. Every aggregation period T , the controller scans the map, computes interval deltas (e.g., retransmission increases from cumulative counters), aggregates RTT samples across flows, and forms normalized buffer-pressure ratios.

Let W_k denote the kernel send-buffer bytes (`sk_wmem_queued`) for flow k , and R_k denote the kernel receive-buffer bytes (`sk_rmem_alloc`) for flow k .

$$\begin{aligned}\rho_t^{\text{snd}} &= \frac{\max_k W_k}{\text{TH_SNDBUF}}, \\ \rho_t^{\text{rcv}} &= \frac{\max_k R_k}{\text{TH_RCVBUF}}.\end{aligned}\quad (1)$$

Interpretation of Eq. 1. We normalize instantaneous buffer pressure into dimensionless ratios in $[0, 1]$ by capping with `TH_SNDBUF` and `TH_RCVBUF`, which act as scaling caps rather than hard limits. We take the maximum over flows ($\max_k$) to be sensitive to the worst offending connection that typically drives tail latency, rather than masking it with an average.

3.3 Adaptive control policy

To avoid reacting to instantaneous jitter, the interval-average RTT (milliseconds) is smoothed by an exponentially weighted moving average (EWMA). Let $\bar{r}_t$ denote the mean of all RTT samples observed during interval t across tracked flows.[♦] The smoothing factor $\alpha \in (0, 1]$ controls the reactivity–stability trade-off: larger α reacts faster but is noisier; smaller α stabilizes but lags. We initialize $\tilde{r}_0$ with the first non-empty $\bar{r}_t$ (or a configured baseline).

$$\tilde{r}_t = \alpha \bar{r}_t + (1 - \alpha) \tilde{r}_{t-1}, \quad \alpha \in (0, 1]. \quad (2)$$

In what follows, we write $\rho_s := \rho_t^{\text{snd}}$ and $\rho_r := \rho_t^{\text{rcv}}$ for brevity, and let Δr_t denote the per-interval increase of the retransmission counter.

Three gates are combined under hysteresis. Let $H > L$ be the high/low RTT bounds, and let $\theta_H > \theta_L$ be the high/low buffer thresholds.

Table 3. Gate conditions at a glance.

Signal	On condition	Off condition
RTT	$\tilde{r}_t > H$	$\tilde{r}_t < L$
Buffer	$\rho_s \geq \theta_H \vee \rho_r \geq \theta_H$	$\rho_s \leq \theta_L \wedge \rho_r \leq \theta_L$
Retrans	$\Delta r_t > 0$	$\Delta r_t = 0$

The RTT gate is evaluated on the smoothed series $\tilde{r}_t$ with hysteresis ($H > L$); the buffer gate uses the dimensionless pressures ρ_s, ρ_r ; the retransmission gate triggers on any positive Δr_t within the interval.

[♦] If no samples arrive in an interval, we carry forward $\tilde{r}_{t-1}$.

For column safety, the intents are stated in rule form:

- Activation intent (want-on): true if RTT-on and (BUF-on or RET-on).
- Deactivation intent (want-off): true if RTT-off and BUF-off and RET-off.

State transitions require persistence to prevent flapping: the controller turns on only if the activation intent holds continuously for $T_{\text{HOLD,ON}}$ seconds, and turns off only if the deactivation intent holds for $T_{\text{HOLD,OFF}}$ seconds. When turning on, actions are selected by severity. Severe pressure (for example, $\tilde{r}_t \gtrsim 100$ ms, $\rho_t^{\text{snd}} \gtrsim 0.9$, or a retransmission observed) triggers a low send rate, larger batch size, and optionally a lower QoS level. Moderate pressure (for example, $\tilde{r}_t \gtrsim 70$ ms or $\rho_t^{\text{snd}} \gtrsim 0.8$) applies a moderate rate limit and batching. Mild pressure adjusts batching only. When turning off, a release command restores publisher defaults. Commands are throttled by a minimum interval to avoid bursts, and the last state is remembered to avoid redundant publications.

3.4 Parameterization and defaults

All thresholds, timers, and endpoints are configured via environment variables at runtime. This avoids overfitting to a single deployment and makes experiments reproducible. Table 2 lists the key parameters and the defaults used in our evaluation.

IV. Experiments

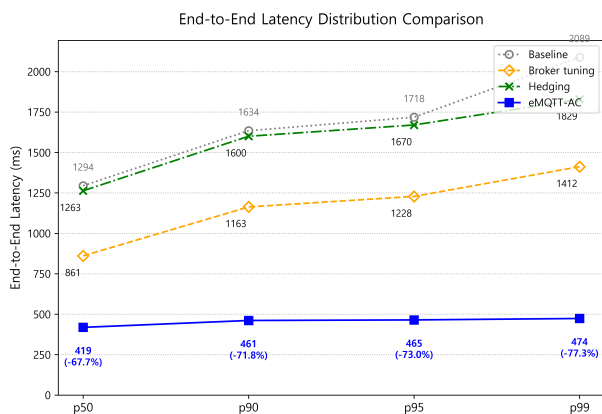


Fig. 1. Latency percentiles (20 publishers, 1,000 msg/s).

4.1 Setup and Method

Our experimental setup features a controller running directly on the broker host, which acts as an edge gateway. This controller utilizes an eBPF agent to observe the normal TCP datapath at the kernel

level by attaching to key functions for retransmissions (`tcp_retransmit_skb`), sending (`tcp_sendmsg`), and receiving (`tcp_rcv_established`). Every two seconds, a user-space component aggregates these signals and publishes control commands to a dedicated MQTT topic, allowing publishers to adapt their behavior without any application code changes. To test this system, we emulated a high-density burst of 1,000 messages per second from 20 publishers to a single subscriber, while simultaneously reproducing long-tail latency conditions by injecting packet loss, delay, and jitter on the broker's uplink using standard Linux facilities. For fair comparison, these impairment intensities were held constant across all runs. We also benchmarked this approach against several alternatives, such as client-side backoff/hedging, broker-level tuning, and MQTT-over-QUIC. Performance was evaluated based on end-to-end latency, the effective message delivery rate, and any resulting CPU overhead. Latency was computed at the subscriber using publisher-embedded timestamps, with a focus on p95/p99 statistics, while the delivered rate measured the effective throughput. The CPU overhead was determined by the resource utilization increase attributable to our agent. To ensure accuracy, each configuration was tested over multiple burst episodes, and all statistics were computed from the system's steady state.

4.2 Results and Analysis

Under the 20-publisher, 1,000 msg/s workload, our proposed method, eMQTT-AC, dramatically improved end-to-end message latency. The mean latency dropped from 1226.7 ms (baseline) to 307.37 ms (-74.9%), with p95 from 1717.9 ms to 464.61 ms (-73.0%) and p99 from 2088.5 ms to 473.50 ms (-77.3%). The standard deviation fell from 402.6 ms to 179.63 ms (-55.4%), indicating far tighter tails and more predictable performance. These gains were achieved while slightly increasing throughput (32.28 → 32.36 msg/s) with only about 2% CPU overhead. These metrics reflect end-to-end application latencies measured at the subscriber; in parallel, the agent-observed kernel `srtt` distribution also contracted, consistent with reduced queuing pressure. To position these results against representative strategies from prior work, we additionally evaluated broker tuning and client-side hedging under the same conditions. As shown in Table 4, eMQTT-AC achieves the most substantial reduction in tail latency while successfully maintaining throughput compared to these alternatives. The system's behavior can be fine-tuned for different

Table 4. End-to-end latency statistics (milliseconds) and throughput across evaluated methods.

Method	p_{50ms}	p_{90ms}	p_{95ms}	p_{99ms}	$mean_{ms}$	std_{ms}	min_{ms}	max_{ms}	throughput (msg/s)
Baseline	1,294.24	1,634.47	1,717.90	2,088.54	1,226.69	402.61	0.01	2,405.55	32.28
Broker tuning (EMQX)	860.57	1,163.31	1,228.33	1,412.13	827.19	316.10	0.01	1,669.88	32.19
Hedging	1,263.02	1,600.14	1,669.90	1,828.90	1,215.46	396.29	0.01	1,940.25	32.17
eMQTT-AC	418.60	461.16	464.61	473.50	307.37	179.63	0.01	502.47	32.36

environments. For example, raising $EWMA_ALPHA$ increases reactivity but risks on-off oscillation unless hold times are also lengthened. A wider hysteresis band between (HI_RTT_US, LO_RTT_US) reduces such flapping at the cost of a slower release from a controlled state. The buffer thresholds (SND_RATIO_HI/LO) further modulate how strongly buffer pressure participates in the control logic. In terms of overhead, the eBPF programs are purely observational and do not alter the datapath. Memory overhead is negligible, and a command cool-down mechanism prevents message bursts. Finally, placing the controller on the broker host confines all changes to a single edge node, significantly simplifying roll-out and management.

V. Conclusion

This paper presented *eMQTT-AC*, an application-transparent, eBPF-based adaptive control system designed to mitigate MQTT tail latency in high-density edge settings. By observing TCP kernel signals (EWMA RTT, retransmissions, and buffer occupancy) and issuing lightweight control commands over a dedicated MQTT topic, eMQTT-AC suppresses incipient queue build-up without modifying client applications or broker internals.

In a workload of 20 publishers producing 1,000 messages per second, eMQTT-AC reduced the 95th percentile end-to-end latency from 1717.9 ms to 464.61 ms (−73.0%) and the 99th percentile end-to-end latency from 2088.5 ms to 473.50 ms (−77.3%). The mean end-to-end latency decreased by 74.9%, and the latency standard deviation decreased by 55.4%. These improvements were achieved with negligible cost: throughput slightly increased (32.28 → 32.36 msg/s) and CPU overhead remained around 2%.

eMQTT-AC complements, rather than replaces, broker or transport-layer innovations. It enables an incremental, broker-agnostic control loop that can be deployed at a single edge node.

Future work will develop a reinforcement learning-based controller that converts kernel-level signals into rate/batch/QoS adjustments to minimize p95/p99 latency while preserving throughput and low overhead.

Overall, eMQTT-AC provides a deployable and lightweight solution for stabilizing MQTT tail latency, making it a promising foundation for reliable and scalable IoT edge deployments.

References

- [1] OASIS Standard, “MQTT Version 5.0,” Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>.
- [2] C.-S. Yeh, S.-L. Chen, and I.-C. Li, “Implementation of mqtt protocol based network architecture for smart factory,” *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, vol. 235, no. 13, pp. 2132–2142, 2021. DOI: 10.1177/09544054211014488. eprint: <https://doi.org/10.1177/09544054211014488>. [Online]. Available: <https://doi.org/10.1177/09544054211014488>.
- [3] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013. DOI: 10.1145/2408776.2408794.
- [4] J. Iyengar and M. Thomson, “QUIC: A UDP-Based Multiplexed and Secure Transport,” Request for Comments, no. 9000, May 2021. DOI: 10.17487/RFC9000.
- [5] M. Rezvani, A. Jahanshahi, and D. Wong, “Characterizing in-kernel observability of latency-sensitive request-level metrics with ebpf,” in *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2024, pp. 24–35. DOI: 10.1109/ISPASS61541.2024.00013.
- [6] Z. Wang, H. Zhang, and Y. Zhang, “An Implementation and Evaluation of MQTT over QUIC,” *IEEE Internet of Things Journal*, vol. 10, no. 15, pp. 13 485–13 498, 2023. DOI: 10.1109/JIOT.2023.3265809.

- [7] Y. Li, Z. Chen, and K. Wang, "A Deep Reinforcement Learning Approach for TCP Congestion Control," in *2022 IEEE International Conference on Communications (ICC)*, 2022, pp. 1–6.
- [8] D. Soni and A. Makwana, "A survey on mqtt: A protocol of internet of things (iot)," Apr. 2017.
- [9] D. Shvaika, A. Shvaika, and V. Artemchuk, "Mqtt broker architectural enhancements for high-performance p2p messaging: Tbmq scalability and reliability in distributed iot systems," *IoT*, vol. 6, no. 3, p. 34, 2025. DOI: [10.3390/iot6030034](https://doi.org/10.3390/iot6030034). [Online]. Available: <https://www.mdpi.com/2624-831X/6/3/34>.
- [10] S. Becker, T. Pfandzelter, N. Japke, D. Bernbach, and O. Kao, *Network emulation in large-scale virtual edge testbeds: A note of caution and the way forward*, 2022. arXiv: [2208.05862](https://arxiv.org/abs/2208.05862) [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2208.05862>.
- [11] L. Rice, *Learning eBPF. Isovalent/Cilium* (Open Book), 2024. [Online]. Available: [https://cilium.isovalent.com/hubfs/Learning-eBPF % 20 - %20Full % 20book.pdf](https://cilium.isovalent.com/hubfs/Learning-eBPF%20-%20Full%20book.pdf).
- [12] Y. Yoo, Z. Niu, C. Yoo, P. Cheng, and Y. Xiong, "Seganet: An advanced iot cloud gateway for performant and priority-oriented message delivery," in *Proceedings of the 7th Asia-Pacific Workshop on Networking*, ser. APNet '23, Hong Kong, China: Association for Computing Machinery, 2023, pp. 54–60, ISBN: 9798400707827. DOI: [10.1145/3600061.3600072](https://doi.org/10.1145/3600061.3600072). [Online]. Available: <https://doi.org/10.1145/3600061.3600072>.
- [13] M. Kempf, B. Jaeger, J. Zirngibl, K. Ploch, and G. Carle, "Quic on the fast lane: Extending performance evaluations on high-rate links," *Computer Communications*, vol. 223, pp. 90–100, 2024, ISSN: 0140-3664. DOI: [10.1016/j.comcom.2024.04.038](https://doi.org/10.1016/j.comcom.2024.04.038). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S014036642400166X>.
- [14] X. Zhang et al., "Quic is not quick enough over fast internet," in *Proceedings of the ACM Web Conference 2024*, ser. WWW '24, Singapore, Singapore: Association for Computing Machinery, 2024, pp. 2713–2722, ISBN: 9798400701719. DOI: [10.1145/3589334.3645323](https://doi.org/10.1145/3589334.3645323). [Online]. Available: <https://doi.org/10.1145/3589334.3645323>.
- [15] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "Bbr: Congestion-based congestion control," *Commun. ACM*, vol. 60, no. 2, pp. 58–66, Jan. 2017, ISSN: 0001-0782. DOI: [10.1145/3009824](https://doi.org/10.1145/3009824). [Online]. Available: <https://doi.org/10.1145/3009824>.
- [16] S. Ha, I. Rhee, and L. Xu, "Cubic: A new tcp-friendly high-speed tcp variant," *SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, Jul. 2008, ISSN: 0163-5980. DOI: [10.1145/1400097.1400105](https://doi.org/10.1145/1400097.1400105). [Online]. Available: <https://doi.org/10.1145/1400097.1400105>.
- [17] M. Alizadeh et al., "Data center tcp (dctcp)," *SIGCOMM Comput. Commun. Rev.*, vol. 40, no. 4, pp. 63–74, Aug. 2010, ISSN: 0146-4833. DOI: [10.1145/1851275.1851192](https://doi.org/10.1145/1851275.1851192). [Online]. Available: <https://doi.org/10.1145/1851275.1851192>.
- [18] S. Wen, R. Han, C. H. Liu, and L. Chen, "Fast drrl-based scheduler configuration tuning for reducing tail latency in edge-cloud jobs," *Journal of Cloud Computing*, vol. 12, Jun. 2023. DOI: [10.1186/s13677-023-00465-z](https://doi.org/10.1186/s13677-023-00465-z).
- [19] S. Dong et al., "Task offloading strategies for mobile edge computing: A survey," *Computer Networks*, vol. 254, p. 110 791, 2024, ISSN: 1389-1286. DOI: [10.1016/j.comnet.2024.110791](https://doi.org/10.1016/j.comnet.2024.110791). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128624006236>.
- [20] P. Sethi and S. R. Sarangi, "Internet of things: Architectures, protocols, and applications," *Journal of Electrical and Computer Engineering*, vol. 2017, no. 1, p. 9 324 035, 2017. DOI: [10.1155/2017/9324035](https://doi.org/10.1155/2017/9324035). eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1155/2017/9324035>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2017/9324035>.